Number-Theoretic Transform in Lean

1. Introduction

The discrete Fourier transform (DFT) of a function taking values in the complex numbers can be generalized to functions taking values in an arbitrary ring R. If we specialize the discrete Fourier transform over a ring to $R = \mathbb{Z}/p\mathbb{Z}$, the integers modulo a prime p, we obtain what is called the number-theoretic transform (NTT).

The fast Fourier transform algorithm (FFT), used to compute the discrete Fourier transform of an n-tuple in $O(n \log n)$ time, can also be applied to the number-theoretic transform. This project is a formalization of both the number-theoretic transform and the fast algorithm to compute it. The advantage of working in $\mathbb{Z}/p\mathbb{Z}$ instead of $\mathbb{C}$, is that all computations can be carried out exactly, making all of our formalization computable.

2. Theory and definitions

Let $x = (x_0, ..., x_{n-1})$ be an n-tuple of elements of $\mathbb{Z}/p\mathbb{Z}$ where p is prime. The NTT of x is obtained by replacing the factors $e^{-i2\frac{\pi}{n}}$ in the definition of the complex DFT by a *primitive n-th root of unity* $\omega \in \mathbb{Z}/p\mathbb{Z}$:

Definition 2.1 (`ntt` in `NTT.lean`): The number-theoretic Transform (NTT) maps x to another n-tuple $y = (y_0, ..., y_{n-1})$ of elements in $\mathbb{Z}/p\mathbb{Z}$ defined by

$$\mathrm{NTT}(x)_k := \sum_{j=0}^{n-1} x_j \omega^{jk}.$$

Definition 2.2: An element $\omega \in \mathbb{Z}/p\mathbb{Z}$ is called a primitive k-th root of unity if $\omega^k = 1$ and $\omega^l = 1 \Rightarrow k \mid l$.

Just like the DFT, the NTT is invertible:

Definition 2.3 (`intt` in `NTT.lean`): The inverse number-theoretic Transform (INTT) maps $y = (y_0, ..., y_{n-1})$ back to the n-tuple

$$\mathrm{NTT}(y)_k := n^{-1} \sum_{j=0}^{n-1} y_j \omega^{-jk}.$$

In order to prove that this is indeed an inverse, we need the following theorem about primitive roots:

Theorem 2.1 (`sum_zpow_mul_eq` in `PrimitiveRoots.lean`): Let $\omega \in \mathbb{Z}/p\mathbb{Z}$ be a primitive n-th root of unity, then for any $m \in \mathbb{Z}$ it holds that

$$\sum_{k=0}^{n-1} \omega^{km} = \begin{cases} n \text{ if } n \mid m \\ 0 \text{ otherwise} \end{cases}.$$

Proof: If n divides m, i.e. there is an integer a such that $m = a \cdot n$, then

$$\sum_{k=0}^{n-1} \omega^{km} = \sum_{k=0}^{n-1} (\omega^n)^{ka} = \sum_{k=0}^{n-1} 1^{ka} = n.$$

If n does not divide m, then by the primitive root property $\omega^m \neq 1$. Hence, we can rewrite the geometric sum to

$$\sum_{k=0}^{n-1} \omega^{km} = \sum_{k=0}^{n-1} (\omega^m)^k = \frac{(\omega^m)^n - 1}{\omega^m - 1} = 0.$$

□

Theorem 2.2 (`left_inv` in `NTT.lean`): Let $n, p \in \mathbb{N}$ and p be prime. If $p \nmid n$, then for any n-tuple $x = (x_0, ..., x_{n-1})$ of elements in $\mathbb{Z}/p\mathbb{Z}$ we have that $\mathrm{INTT}(\mathrm{NTT}(x)) = x$.

Proof: Since p does not divide n, n is non-zero in $\mathbb{Z}/p\mathbb{Z}$ and hence its inverse n^{-1} is well defined.

$$\begin{aligned}
\mathrm{INTT}(\mathrm{NTT}(x))_j &= n^{-1} \sum_{k=0}^{n-1} \left(\sum_{l=0}^{n-1} x_l \omega^{lk} \right) \omega^{-jk} \\
&= n^{-1} \sum_{l=0}^{n-1} x_l \left(\sum_{k=0}^{n-1} \omega^{(l-j)k} \right)
\end{aligned}$$

Notice that $k \mid l - j$ if and only if $l = j$, hence

$$\begin{aligned}
&= n^{-1} \sum_{l=0}^{n-1} x_l n \\
&= x_j,
\end{aligned}$$

by Theorem 2.1. □

The proof of the other inversion direction, namely that $\mathrm{NTT}(\mathrm{INTT}(x)) = x$, is entirely analogous and follows the same steps as above.

2.1. Convolution Theorem

For n-tuples of elements in $\mathbb{Z}/p\mathbb{Z}$ we have the following notion of convolution:

Definition 2.1.1 (`convolution` in `Convolution.lean`): Let $x = (x_0, ..., x_{n-1})$ and $y = (y_0, ..., y_{n-1})$ be n-tuples of elements in $\mathbb{Z}/p\mathbb{Z}$, the *circular convolution* of x and y is the n-tuple defined by

$$(x \star y)_k = \sum_{j=0}^{n-1} x_j y_{(k-j) \bmod n}.$$

The convolution theorem states that the convolution in the time domain corresponds to pointwise multiplication in the frequency domain:

Theorem 2.1.1 (`ntt_convolution` in `NTT.lean`): Let ω be a primitive n-th root of unity in $\mathbb{Z}/p\mathbb{Z}$, then

$$\mathrm{NTT}(x \star y) = \mathrm{NTT}(x) \cdot \mathrm{NTT}(y),$$

where $\cdot$ denotes the pointwise multiplication of two n-tuples.

Proof: Let $z = x \star y$. By the definition of the NTT and the cyclic convolution we have

$$\begin{aligned}\mathrm{NTT}(z)_k &= \sum_{m=0}^{n-1} \left(\sum_{j=0}^{n-1} x_j y_{(m-j) \bmod n} \right) \omega^{mk} \\ &= \sum_{j=0}^{n-1} x_j \sum_{m=0}^{n-1} y_{(m-j) \bmod n} \omega^{mk}.\end{aligned}$$

We substitute $l = (m - j) \bmod n$. As m ranges from 0 to $n-1$ so does l. Also notice that $\omega^n = 1$, hence $\omega^{mk} = \omega^{(l+j)k} = \omega^{lk}\omega^{jk}$.

$$\begin{aligned}&= \sum_{j=0}^{n-1} x_j \sum_{l=0}^{n-1} y_l \omega^{lk} \omega^{jk} \\ &= \left(\sum_{j=0}^{n-1} x_j \omega^{jk} \right) \left(\sum_{l=0}^{n-1} y_l \omega^{lk} \right) \\ &= \mathrm{NTT}(x)_k \cdot \mathrm{NTT}(y)_k.\end{aligned}$$

□

A similar result holds for the inverse NTT:

Theorem 2.1.2 (`intt_convolution` in `NTT.lean`): Let ω be a primitive n-th root of unity in $\mathbb{Z}/p\mathbb{Z}$, then

$$\mathrm{INTT}(x \star y) = n \cdot \mathrm{NTT}(x) \cdot \mathrm{NTT}(y).$$

Proof: This follows directly from the fact that if ω is a primitive n-th root of unity, then so is ω^{-1}, and the previous theorem. □

Furthermore, for any x, y we have that
- $\mathrm{NTT}(x) \star \mathrm{NTT}(y) = n \cdot \mathrm{NTT}(x \cdot y)$, see `convolution_ntt` in `NTT.lean`, and
- $\mathrm{INTT}(x) \star \mathrm{INTT}(y) = \mathrm{NTT}(x \cdot y)$, see `convolution_intt` in `NTT.lean`.

3. Fast Algorithms

Computing the NTT naively using the definition requires $O(n^2)$ operations in $\mathbb{Z}/p\mathbb{Z}$. However, just like the DFT, the NTT can be computed in $O(n \log n)$ time using a divide-and-conquer approach.

The existence of such an algorithm follows almost immediately from the following decomposition of the NTT: Let x be a vector in $(\mathbb{Z}/p\mathbb{Z})^n$ where n is even and let ω be a primitive n-th root of unity in $\mathbb{Z}/p\mathbb{Z}$, for any $k = 0, \ldots, n-1$ we have

$$(\mathrm{NTT}(x))_k = \sum_{j=0}^{n-1} x_j \omega^{jk} = \sum_{j=0}^{n/2-1} x_{2j} (\omega^2)^{jk} + \omega^k \sum_{j=0}^{n/2-1} x_{2j+1} (\omega^2)^{jk}.$$

The right-hand side of the above equation resembles two NTTs of length $\frac{n}{2}$: one for the even-indexed elements of x and one for the odd-indexed elements, using ω^2 as the root of unity. It is easy to verify that if ω is a primitive n-th root of unity, then ω^2 is a primitive $(\frac{n}{2})$-th root of unity.

A small caveat is that the index k in the original problem ranges from 0 to $n-1$, while the sub-NTTs on the right-hand side produce vectors defined only for indices from 0 to $\frac{n}{2}-1$. However, we can resolve this by observing that the sub-problems are periodic with period $\frac{n}{2}$

$$(\omega^2)^{j(k \bmod \frac{n}{2})} = (\omega^2)^{j\left(k - \frac{n}{2}\left\lfloor \frac{k}{n/2} \right\rfloor\right)} = (\omega^2)^{jk} (\omega^n)^{-j\left\lfloor \frac{k}{n/2} \right\rfloor} = (\omega^2)^{jk}.$$

Hence, we obtain the following recursive formula for the NTT:

$$\mathrm{NTT}(\omega, x)_k = \mathrm{NTT}(\omega^2, x^{\mathrm{even}})_{k \bmod \frac{n}{2}} + \omega^k \cdot \mathrm{NTT}(\omega^2, x^{\mathrm{odd}})_{k \bmod \frac{n}{2}},$$

where $x^{\mathrm{even}} = (x_0, x_2, ..., x_{n-2})$ and $x^{\mathrm{odd}} = (x_1, x_3, ..., x_{n-1})$. By evaluating the sub-NTTs at $k \bmod \frac{n}{2}$, we save ourselves from having to recompute the two half-length NTTs for the indices $k = \frac{n}{2}, ..., n-1$.

Assuming that $n = 2^l$ for some $l \in \mathbb{N}$ and keeping in mind that $\omega^{k+\frac{n}{2}} = -\omega^k$ we obtain the following recursive algorithm for computing the NTT, called the Fast Number-Theoretic Transform (FNTT):

```
1 function FNTT(ω, x)
2     match l with
3         0 → x
4         l + 1 →
5             x^even ← (x_0, x_2, ..., x_{2^l − 2}),   x^odd ← (x_1, x_3, ..., x_{2^l − 1})
6             y^even ← FNTT(ω^2, x^even),   y^odd ← FNTT(ω^2, x^odd)
7             for k = 0..., 2^l − 1 do
8                 y_k ← y_k^even + ω^k · y_k^odd,   y_{k+2^l} ← y_k^even − ω^k · y_k^odd
9             return y
```

Figure 1: Pseudocode for the Fast Number-Theoretic Transform (FNTT) of a vector of length $n = 2^l$.

The above algorithm runs in $O(n \log n)$ time, as each level of recursion requires $O(2^l)$ operations to combine the two half-length NTTs, and there are $l = \log_2(n)$ levels of recursion.

4. Formalization in Lean

4.1. NTT definition properties

The statements in this section are in the file `NTT.lean`, for this entire section assume the variables

```
variable {n p : ℕ} [Fact p.Prime]
```

We defined NTT and INTT the following way:

```
def ntt (ω : ZMod p) : (Fin n → ZMod p) → (Fin n → ZMod p) :=
  fun x k ↦ ∑ j, x j * ω ^ ((j * k) : ℤ)
```

```
def intt_aux (ω : ZMod p) : (Fin n → ZMod p) → (Fin n → ZMod p) :=
  fun x k ↦ ∑ j, x j * ω ^ (-(j * k : ℤ))

def intt (ω : ZMod p) (x : Fin n → ZMod p) : (Fin n → ZMod p) :=
  (n : ZMod p)⁻¹ • intt_aux ω x
```

Defining tuples as functions from `Fin n` is the standard way to represent fixed-length tuples in mathlib, this allow us to us to invoke various lemmas about `Fin n` and sums over `Fin n` from mathlib.

Theorem names	Description
`intt_as_ntt`	The INTT is an NTT with the inverse root of unity, scaled by n^{-1}.
`ntt_add`, `ntt_smul`	Linearity of the NTT
`intt_add`, `intt_smul`	Linearity of the inverse transform
`ntt_shift`	Frequency shifting property, $(\mathrm{NTT}_\omega(x))_{i-j} = \left(\mathrm{NTT}_\omega\left(\omega^{-jk}x_k\right)\right)_i$
`left_inv`, `right_inv`	INTT is the inverse of the NTT assuming `p` does not divide `n` and `ω` is a primitive `n`-th root of unity
`(i)ntt_convolution`, `convolution_(i)ntt`	Convolution theorems relating NTT/INTT of convolutions to pointwise multiplications.

4.2. Primitive roots of unity

The theory of primitive roots is well developed in mathlib, the missing results we need are in the file `PrimitiveRoots.lean`. This file contains the fact that if `ω` is a primitive `n`-th root of unity in `Zmod(p)`, then ω^2 is a primitive `n/2`-th root of unity, as well as Theorem 2.1

4.3. FNTT on Vectors

In order to get an FFT algorithm that runs in $O(N \log N)$ time, where $N = 2^n$, we need to work with an appropriate data structure, which allows us to reuse computations. We chose to work with `Vector` which is a wrapper around fixed-length arrays in Lean. The implementation of the FNTT is in the file `FNTT.lean`. The main function is `Vector.fntt`, which implements the pseudocode given in Figure 1:

```
def Vector.fntt {p : ℕ} [Fact (Nat.Prime p)] {n : ℕ} (xs : Vector (ZMod p) (2 ^ n))
(ω : ZMod p) : Vector (ZMod p) (2 ^ n) := fntt_aux xs ω (getPowers ω n)
  where fntt_aux {n : ℕ} (xs : Vector (ZMod p) (2 ^ n)) (ω : ZMod p)
    (powers : Vector (ZMod p) (2 ^ n)) : Vector (ZMod p) (2 ^ n) :=
  match n with
  | 0 => xs
  | n + 1 =>
    let powers' := powers.restrictEven -- [ω ^ 0, ω ^ 2, ω ^ 4, ...]
    let ws := (powers.extract 0 (2 ^ n)).cast (...) -- [ω ^ 1, ω ^ 2, ..., ω ^ (2^n)]

    let y_even := fntt_aux xs.restrictEven (ω ^ 2) powers'
    let y_odd  := fntt_aux xs.restrictOdd (ω ^ 2) powers'

    let left := zipWith3 (fun e o w ↦ e + w * o) y_even y_odd ws
    let right := zipWith3 (fun e o w ↦ e - w * o) y_even y_odd ws

    (left ++ right).cast (Eq.symm (Nat.two_pow_succ n))
```

The function `getPowers` is a helper function that precomputes the powers of ω needed in the algorithm to avoid recomputing them multiple times, it computes the vector $(\omega^0, \omega^1, \omega^2, ..., \omega^{2^n-1})$, iteratively in $O(2^n)$ time.

4.4. Correctness of the FNTT

We consider the function `Vector.fntt` to be correct if it commutes with the function `Vector.get` which converts a `Vector` to a function from `Fin n`, that is if `(xs.fntt ω).get = ntt ω xs.get` for any `xs : Vector (ZMod p) (2 ^ n)`.

We do not prove this result directly, but rather we introduce a recursive definition of the NTT on functions `Fin (2 ^ n) → Zmod p`, called `ntt_rec`.

```
def ntt_rec (ω : ZMod p) (x : Fin (2 ^ n) → ZMod p) (k : Fin (2 ^ n)) : ZMod p :=
  match n with
  | 0 => x k
  | n + 1 =>
    let k' := Fin.ofNat (2 ^ n) k
    let y_even := ntt_rec (ω ^ 2) (restrictEven x) k'
    let y_odd  := ntt_rec (ω ^ 2) (restrictOdd x)  k'

    y_even + (ω ^ (k : ℕ)) * y_odd
```

This function does not compute the NTT in $O(n2^n)$ time, since it only computes the NTT recursively pointwise without reusing any computations. However, its definition mirrors the structure of the FNTT algorithm closely enough that we can prove the following two theorems by induction on `n`.

```
theorem ntt_rec_eq_ntt (h : IsPrimitiveRoot ω (2 ^ n)) (x : Fin (2 ^ n) → ZMod p) :
  ntt_rec ω x = ntt ω x

lemma vector_fntt_eq_ntt_rec (hω : IsPrimitiveRoot ω (2 ^ n)) :
  ntt_rec ω xs.get = (xs.fntt ω).get
```

The main ingredient for proving the recursive NTT coincides with the standard NTT definition is the decomposition of the NTT into even and odd parts, which follows from equivalence `finTwoPowSuccEquiv : Fin (2 ^ n) ⊕ Fin (2 ^ n) ≃ Fin (2 ^ (n + 1))` in the file `Aux.lean`.

To prove the vector FNTT coincides with the recursive NTT, the key is to show that the helper functions `restrictEven` and `restrictOdd` on functions correspond to the vector operations `Vector.restrictEven` and `Vector.restrictOdd`, respectively. This is done in the lemmas `restrictEven_of_vector` and `restrictOdd_of_vector` in the file `Vector.lean`.

The actual correctness theorem of the FNTT the follows immediately and only requires one line of proof:

```
theorem Vector.fntt_correct (h : IsPrimitiveRoot ω (2 ^ n)) :
    (xs.fntt ω).get = ntt ω xs.get :=
  Eq.trans (vector_fntt_eq_ntt_rec xs h).symm (ntt_rec_eq_ntt h xs.get)
```

4.5. Running time analysis

The running time analysis is located in the file `RunningTime.lean`, it includes a copy of the FNTT algorithm called `Vector.fnttT` which follows the definition of `Vector.fntt` closely, but uses the time monad from class.

```
def Vector.fnttT (ω : ZMod p) : TimeM (Vector (ZMod p) (2 ^ n)) := do
  let powers ← getPowersT ω n
  fntt_auxT xs ω powers
```

```
where fntt_auxT {n : ℕ} (xs : Vector (ZMod p) (2 ^ n)) (ω : ZMod p)
  (powers : Vector (ZMod p) (2 ^ n)) : TimeM (Vector (ZMod p) (2 ^ n)) :=
match n with
| 0 => return xs
| n + 1 => do
  let powers' ← powers.restrictEvenT

  let ws ← (powers.extractT 0 (2 ^ n)) >>= castT ...

  let xs_even ← xs.restrictEvenT
  let xs_odd ← xs.restrictOddT

  let y_even ← fntt_auxT xs_even (ω ^ 2) powers'
  let y_odd  ← fntt_auxT xs_odd (ω ^ 2) powers'

  let left ← zipWith3T (fun e o w ↦ e + w * o) y_even y_odd ws
  let right ← zipWith3T (fun e o w ↦ e - w * o) y_even y_odd ws

  appendT left right >>= Vector.castT (Eq.symm (Nat.two_pow_succ n))
```

In the file `ComputationModel.lean` we wrap the basic vector operations in the time monad. For most vector operations operating on vectors of length n, we assume a running time of n. The exception is `Vector.castT` which represents a type-level cast and is assumed to be free.

Function	**Description**	**Cost**
`Vector.castT`	Cast vector length given equality proof	Free
`Vector.extractT`	Extract a sub-vector, copies the array	$O(n)$
`Vector.appendT`	Concatenate two vectors[1]	$O(n)$
`Vector.restrictEvenT`	Collect elements at even indices, copies the array	$O(n)$
`Vector.restrictOddT`	Collect elements at odd indices, copies the array	$O(n)$
`Vector.zipWith3T`	Apply function to 3 vectors pointwise	$O(n)$
`Vector.mulT`	Pointwise multiplication of two vectors	$O(n)$

Table 1: Assumed costs for primitive vector operations in `TimeM`

Under these assumptions, we can prove that the running time of `Vector.fnttT` is $(4n + 1)2^n$, as shown in `Vector.fnttT_time`. The $+1$ term in the cost factor corresponds to the cost introduced by the computation of `getPowers`, as proven in `getPowersT_time`. The factor 4 could be improved by avoiding copying arrays unnecessarily, e.g. in `restrictEvenT` and `restrictOddT` and using views instead, however this cannot be done using the `Vector` type and we would have to resort to raw arrays.

Finally, we analyze the complexity of the full convolution algorithm:

[1]Running time as per the Lean Reference Manual: https://lean-lang.org/doc/reference/latest/Basic-Types/Arrays/#Array___append

Theorem 4.5.1: The running time of `Vector.fastConvolutionT` for vectors of size 2^n is $(12n + 4)2^n$.

Proof: The fast convolution involves:
- Two forward FNTTs: $2 \cdot (4n + 1)2^n$.
- One pointwise multiplication: 2^n.
- One inverse FNTT (same cost as forward): $(4n + 1)2^n$.

Summing these gives $3(4n + 1)2^n + 2^n = (12n + 3)2^n + 2^n = (12n + 4)2^n$. This confirms the $O(N \log N)$ complexity where $N = 2^n$. □

5. Conclusion

In this project, we successfully formalized the Number-Theoretic Transform and the Fast Number-Theoretic Transform algorithm in Lean 4. We proved the correctness of the algorithms and verified their asymptotic running time. By working over $\mathbb{Z}/p\mathbb{Z}$, all our definitions are executable, allowing us to run the algorithms on concrete examples as shown in `Examples.lean`.

One limitation of our formalization is the use of the `Vector` type, which ensures length safety but often requires copying data when performing split operations like restrictEven or extract. As noted in the running time analysis, this introduces a constant factor overhead (the factor of 4 in the FNTT cost). Using Array views or indices directly could eliminate this overhead, but would significantly complicate the formalization of correctness and length invariants.

Despite this, the asymptotic complexity of $O(N \log N)$ (where $N = 2^n$) is preserved, demonstrating the efficiency of the formalized algorithm.